

Fazit & Reflexion

WebLab Projekt -- Projektanalyse, Lessons Learned und persönliche Reflexion

Autor(en):	Vorname Nachname (vorname.nachname@stud.hslu.ch) – Matrikelnr.: XX-XXX-XXX
Modul:	WebLab (FS 2026)
Dozierende/Betreuung:	Dozent Name
Version:	1.0
Datum:	31.08.2026

Management Summary / Abstract

Dieses Dokument fasst die Ergebnisse des WebLab-Projekts zusammen, vergleicht die Anforderungen mit dem erreichten Stand (Soll-Ist-Vergleich), reflektiert technische sowie methodische Erkenntnisse und gibt einen Ausblick auf mögliche zukünftige Erweiterungen.

Inhaltsverzeichnis

1	Zusammenfassung des Projekts	2
1.1	Projektüberblick & Ausgangslage	2
1.2	Wichtigste Projektergebnisse	2
2	Zielerreichung & Soll-Ist-Vergleich	2
2.1	Funktionaler Soll-Ist-Vergleich	2
2.2	Nicht-funktionale Anforderungen	3
3	Erkenntnisse & Lessons Learned	3
3.1	Technische Erkenntnisse	3
3.2	Aufgetretene Herausforderungen & Lösungen	3
4	Persönliche & Methodische Reflexion	3
4.1	Vorgehensweise & Arbeitsmethodik	3
4.2	Kompetenzgewinn	4
5	Ausblick & Weiterentwicklung	4
5.1	Potenzielle Erweiterungen	4
5.2	Fazit	4

1 Zusammenfassung des Projekts

1.1 Projektüberblick & Ausgangslage

TODO

Fassen Sie hier das Projekt in wenigen Absätzen zusammen: Was wurde gebaut, wer waren die Hauptnutzer und wie verlief die Entwicklung?

Im Modul WebLab wurde eine vollwertige Webapplikation konzipiert, architektonisch aufgebaut und implementiert. Das Hauptaugenmerk lag auf der Beherrschung des modernen Web-Stacks, der sauberen Trennung von Frontend, Backend und Persistenzschicht sowie der Einhaltung hoher Code-Qualitätsstandards.

1.2 Wichtigste Projektergebnisse

- **Funktionsfähige Webanwendung:** Vollständige Implementierung der Kern-User-Stories (Frontend-SPA und Backend-REST-API).
- **Robuste Architektur:** Modulare Bausteinstruktur mit klar definierten Schnittstellen, State-Management und Validierung.
- **Umfassende Dokumentation:** Vollständige Architekturdokumentation (inkl. ADRs), lückenloses Arbeitsjournal und diese Reflexion.
- **Automatisierung & Tooling:** Schnelle Build- und Test-Workflows mit Typst-Dokumentationspipeline.

2 Zielerreichung & Soll-Ist-Vergleich

2.1 Funktionaler Soll-Ist-Vergleich

Die nachfolgende Tabelle vergleicht die zu Beginn definierten Anforderungen mit dem tatsächlich erreichten Umsetzungsstand:

Anforderung	Soll-Beschreibung	Status	Ist-Zustand / Bemerkung
F01 - Auth	Registrierung, Login, JWT-Generierung	Erfüllt	Vollständig implementiert inkl. Validierung und Secure Token Handling.
F02 - CRUD	Erstellen, Bearbeiten, Löschen und Anzeigen der Kernobjekte	Erfüllt	Vollständig mit Bestätigungsdialogen und Optimistic UI Updates.
F03 - Filter/Suche	Dynamische Filterung & Volltextsuche im Frontend	Erfüllt	Debounced Search mit client-seitigem Caching.
F04 - Responsive UI	Nutzbarkeit auf Desktop, Tablet und Smartphone	Erfüllt	Responsive Breakpoints getestet auf gängigen Bildschirmgrößen.
F05 - Zusatzfeature	Erweitertes Feature (z. B. Export, Dark Mode, Statistiken)	Teilweise	Grundversion vorhanden, weitere Visualisierungen als Ausblick.

2.2 Nicht-funktionale Anforderungen

- **Performance:** Ladezeiten liegen deutlich unter den anvisierten Grenzwerten. Initial Load < 1.2s.
- **Code-Qualität:** TypeScript Strict Mode ohne Typfehler, saubere Formatierung über Linter.
- **Testabdeckung:** Wichtigste Business-Services und Controller sind mit Unit-Tests abgedeckt.

3 Erkenntnisse & Lessons Learned

3.1 Technische Erkenntnisse

Im Verlauf des Projekts wurden wertvolle technische Erfahrungen gesammelt:

- **TypeScript im Fullstack-Umfeld:** Die gemeinsame Nutzung von Interfaces/DTOs zwischen Frontend und Backend reduzierte Schnittstellenfehler massiv und beschleunigte die Entwicklung.
- **Zustandsverwaltung (State Management):** Ein klares unidirektionales Datenfluss-Muster verhinderte Race Conditions und Inkonsistenzen bei asynchronen API-Aufrufen.
- **Effiziente Toolchains:** Moderne Build-Tools (z. B. Vite / Angular CLI / Typst) verkürzten Feedback-Schleifen im Vergleich zu älteren Setups erheblich.

3.2 Aufgetretene Herausforderungen & Lösungen

Herausforderung	Problemursache	Ergriffene Lösung
CORS & Auth-Cookies	Unterschiedliche Ports bei lokaler Entwicklung für Frontend (4200/5173) und Backend (3000).	Einrichtung eines Reverse Proxys in der Dev-Konfiguration sowie korrekte <code>Access-Control-Allow-Credentials</code> Header.
Asynchrones State-Handling	UI flackerte bei schnellen aufeinanderfolgenden Filter-Eingaben.	Einsatz von Debouncing (300ms) und Abbrechen noch laufender HTTP-Requests (AbortController).
Komplexe Datenbankabfragen	N+1 Abfrageproblem bei verschachtelten Entitäten.	Optimierung der ORM-Abfragen via Eager-Loading bzw. gezielte <code>JOIN</code> -Statements.

4 Persönliche & Methodische Reflexion

4.1 Vorgehensweise & Arbeitsmethodik

TODO

Reflektieren Sie hier über Ihre Arbeitsweise: Wie gut funktionierte die Planung, das Zeitmanagement und die Priorisierung?

Die Entwicklung folgte einem iterativen, agilen Ansatz mit kurzen Entwicklungszyklen. Dies ermöglichte ein frühzeitiges Testen von Prototypen und ein schnelles Reagieren auf unvorhergesehene technische Hürden.

- **Was funktionierte sehr gut:**
 - Frühes Festlegen der Datenstrukturen und API-Verträge.
 - Kontinuierliches Führen des Arbeitsjournals direkt nach jeder Arbeitssitzung.
 - Konsequente Nutzung von Feature-Branches und aussagekräftigen Commit-Messages.
- **Verbesserungspotenzial für künftige Projekte:**
 - Noch detailliertere Zeitschätzungen für komplexe UI-Animationen und Edge-Case-Fehlerbehandlung einplanen.
 - Automatisierte End-to-End-Tests (z. B. Playwright / Cypress) noch früher im Projektverlauf etablieren.

4.2 Kompetenzgewinn

Durch die intensive praktische Arbeit im WebLab konnten wesentliche Kompetenzen vertieft werden:

1. Konzeption und Implementierung skalierbarer Webarchitekturen.
2. Praktische Anwendung von Sicherheitsstandards im Web (OWASP, JWT, CORS).
3. Professionelle Dokumentation technischer Systeme mit Typst.

5 Ausblick & Weiterentwicklung

5.1 Potenzielle Erweiterungen

Für eine zukünftige Weiterentwicklung der Webanwendung bieten sich folgende Ausbaustufen an:

1. **Echtzeit-Synchronisation (WebSockets / SSE):** Live-Aktualisierungen bei gleichzeitiger Bearbeitung durch mehrere Benutzer.
2. **Progressive Web App (PWA) Support:** Offline-Fähigkeit durch Service Worker und Caching-Strategien.
3. **Erweiterte Analytics & Dashboard:** Visualisierung von Nutzungsstatistiken mit Diagrammen (z. B. Chart.js oder D3).
4. **Mehrsprachigkeit (i18n):** Internationalisierung der Benutzeroberfläche in Deutsch, Englisch und Französisch.

5.2 Fazit

Das Projekt WebLab konnte erfolgreich abgeschlossen werden. Die gesteckten funktionalen und nicht-funktionalen Anforderungen wurden erfüllt und die entwickelte Architektur bietet eine solide, zukunftsfähige Basis für künftige Erweiterungen.